

# MATH 231 – Linear Algebra I

Extra Credit 1 ■ NumPy for Homework 2

Kiely Hall 326 ■ Instructor: Kevin Bedoya

**Due:** to be announced on the class Discord.    **Worth:** to be announced on the class Discord.

**This assignment is optional.** It earns extra credit and nothing else: skipping it costs you nothing, and no later assignment depends on it. It is the programming companion to Homework 2 — almost every problem checks, or takes further, a problem you worked by hand there.

**Problem numbers.** A number with a letter — E.1, B.2, F.6(a) — refers to **Homework 2**. A bare number — Problem 3, Problem 7(c) — refers to this assignment.

**Grading.** Every problem is worth the same number of points. The assignment is graded by running `runec1.py` and reading what it prints — on whether it produces correct answers, not on programming style.

**Use it to check Homework 2.** Every numerical answer on Homework 2 can be verified in a few lines of NumPy, and you are encouraged to check all of them before submitting — that is what the tool is for. Two limits. The hand work on Homework 2 must still be shown: a correct number with no derivation earns partial credit at best. And no proof can be checked this way, since code confirms a statement only for the numbers you tried, not for all of them.

**Collaboration** is encouraged — work at a board together, argue about the problems. What you submit must be written by you, in your own code.

## What to submit

Push two Python files to your own MATH 231 GitHub repository, in a folder named `extra-credit/ec1/`:

- `ec1.py` — your work, one function per question part;
- `runec1.py` — which imports those functions and runs them all, labelling each answer.

Do not submit a copy of your output: `runec1.py` is **run as submitted**, and the assignment is graded from what it prints. *How to structure your submission*, below, specifies both files in full. The rules are the same as on Homework 1.

**Python is the only language accepted.** Any other language earns no credit, however correct it is.

If you have not set up the repository yet, Setup Guide 1 (§“Create your folders” and §“Save your work and send it to GitHub”) is the walkthrough. Only pushed work can be graded.

## Label every answer

The labelling is built into the file structure specified under *How to structure your submission*: one function per question part in `ec1.py`, named after that part, with a comment naming it directly above; and one labelled line per part in `runec1.py`. Code that runs correctly but cannot be matched to a question is graded the same way an unlabelled answer sheet is.

## What is new since Homework 1

This assignment assumes Homework 1's Part F: arrays, +, -, scaling, `np.linalg.norm`, `np.isclose`, `np.allclose`, and the two-file submission. The short tutorial below covers only what is new — vectors of any length, the dot product, the other norms, angles, and functions of one and two variables. Then comes the submission specification (the same rules as before, with the file names changed), then Problems 1–9. The **Python Recipe Book** ([lectures/python-recipe/](#)) covers the vector calls in its §2 and floating-point behaviour in its §7; the remaining calls are introduced below or in the problem that uses them.

### Vectors in $\mathbb{R}^n$

Nothing changes when a vector has more than two entries.

```
import numpy as np

u = np.array([1.0, -2.0, 0.0, 3.0])  # a vector in  $\mathbb{R}^4$ 
len(u)                             # 4 -- the  $n$  in  $\mathbb{R}^n$ 

np.zeros(5)                         # array([0., 0., 0., 0., 0.])
np.ones(5)                          # the all-ones vector  $\mathbf{1}$  in  $\mathbb{R}^5$ 
np.arange(1, 6)                     # array([1, 2, 3, 4, 5])

e3 = np.zeros(5)                    # the standard basis vector  $e_3$  in  $\mathbb{R}^5$ :
e3[2] = 1.0                         # slot 3 is index 2
```

**Watch out.** Adding vectors of different lengths raises an error, exactly as in lecture: `np.ones(3) + np.ones(4)` fails with `ValueError`.

### The dot product

```
v = np.array([2.0, 1.0, -1.0, 0.0])

u @ v          # 0.0 -- the dot product  $\langle u, v \rangle$ 
np.dot(u, v)   # 0.0 -- the same number
u @ u          # 14.0 --  $\langle u, u \rangle = \|u\|^2$ 
```

**Watch out.** `u * v` is still *not* the dot product. It multiplies entry by entry and returns a vector. `np.sum(u * v)` does equal `u @ v`, but write `u @ v`: it says what you mean.

### The other norms, and a few reductions

```
np.linalg.norm(u)          # 3.7416...   $\|u\|_2 = \sqrt{14}$ 
np.linalg.norm(u, ord=1)   # 6.0         $\|u\|_1$ 
np.linalg.norm(u, ord=np.inf) # 3.0       $\|u\|_{\infty}$ 
np.linalg.norm(u, ord=3)   #  $\|u\|_3$ 

np.abs(u)                  # array([1., 2., 0., 3.]) -- entrywise  $|u_i|$ 
np.sum(u)                   # 2.0 -- the sum of the entries
np.max(np.abs(u))          # 3.0 -- the largest  $|u_i|$ 
```

```
np.mean(u)                # 0.5 -- the average entry
```

## Angles

```
c = (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v)) # cos(u, v)
theta = np.arccos(np.clip(c, -1.0, 1.0))                # radians, in [0, pi]
np.degrees(theta)                                       # 90.0
```

`np.clip` guards against rounding pushing `c` a hair outside  $[-1, 1]$ , where `np.arccos` is undefined and returns `nan`.

## Functions of one and two variables

A mathematical function becomes a Python function with `def`. NumPy supplies the elementary functions.

```
def f(x, y):
    return x**3 - 3*x*y + y**2

f(1.0, 2.0)                # -1.0

np.exp(1.0)                 # 2.718281828...   e^1
np.log(2.0)                 # 0.693147...     the NATURAL log, ln 2
np.sin(np.pi / 6)          # 0.4999999...    sin(pi/6)
np.sqrt(9.2)               # 3.03315...
```

**Watch out.** `np.log` is  $\ln$ . The base-10 logarithm is `np.log10`; you will not need it.

A function can be handed to another function, just like a number. That is how Problem 7 differentiates *any*  $f$  with one piece of code:

```
def twice(g, x):
    return 2 * g(x)

twice(np.exp, 0.0)         # 2.0
```

## Loops over step sizes, and random vectors

```
hs = 10.0 ** -np.arange(1, 16)    # 1e-1, 1e-2, ..., 1e-15
for h in hs:
    print(h, (np.exp(h) - 1) / h)

rng = np.random.default_rng(231)   # a fixed seed, as in Homework 1
x = rng.normal(size=10)            # a random vector in  $\mathbb{R}^{10}$ 
```

## How to structure your submission

The assignment is submitted as **two** Python files with different jobs, exactly as on Homework 1.

- **ec1.py — your work.** One function per lettered part, named *q number letter*: Problem 2(a) becomes q2a, Problem 7(c) becomes q7c. Each function **returns** its answer; when an answer has several pieces, return them together as a tuple or a list. Helper functions (*my\_dot*, *pnorm*, *proj*, ...) are defined in this file too, and may be shared between parts.
- **runec1.py — the driver.** It imports those functions from *ec1.py* and calls them in order, one line per part, printing a label before each answer. It contains no mathematics of its own. **This is the file that is run to grade the assignment.**

**ec1.py — definitions only.** Nothing may run when the file is imported: define your functions and stop. A `print` sitting at the left margin fires the moment *runec1.py* imports the file, and its output lands above the labels, where it cannot be matched to a question.

```
# ec1.py -- the work.  Definitions only; nothing runs on import.

import numpy as np

def pnorm(v, p):                                     # a helper, used by several parts
    ...

# --- 2(c) -----
def q2c():
    v = np.array([-3.0, 4.0])
    return [pnorm(v, p) for p in (1, 2, 3, 4, 10, 100)] + [infnorm(v)]
```

**runec1.py — one line per part.** Import the functions, then call them in the order the questions appear. Each line prints a label followed by the answer that call produced:

```
# runec1.py -- runs every question in order.  No mathematics in this file.

import ec1

print("Output of question 1 part a : ", ec1.q1a())
print("Output of question 1 part b : ", ec1.q1b())
print("Output of question 2 part a : ", ec1.q2a())
```

The label is required, and its wording is fixed:

"Output of question <number> part <letter> : "

So Problem 7(c) is labelled `Output of question 7 part c : .`

**Watch out.** Several parts below ask you to “answer in a comment”. Those comments go in *ec1.py*, directly under the function for that part, and are read by the grader. A part that asks *only* for a comment still gets its function and its labelled line: define it to return the string “see comment”. Everything else is *returned*, not printed: *runec1.py* does all the printing.

**Watch out.** Problems 2, 3, 4, 6 and 8 use random numbers. Create the generator *inside* the function that uses it, with the seed given, so that every run of *runec1.py* prints the same thing.

**Checkpoint.** Run *runec1.py* once before you push and read the output top to bottom. Every part of every question should appear, in order, under its own label, with no error and nothing unlabelled.

## The problems

**Problem 1** (Checking Homework 2, Part E).

- (a) Enter `u`, `v`, `w` of Problem E.1 and return the values asked for in every one of its parts (a)–(g), in order, as one list. The angle in (e) should be in degrees.
- (b) For the data `x` and `y` of Problem E.5, return: `proj1(x)`; the deviation vector; its dot product with `1`;  $\sigma^2$  and  $\sigma$ ; and  $r$ . Then return, alongside your own values, `np.mean(x)`, `np.var(x)`, `np.std(x)` and `np.corrcoef(x, y)[0, 1]`, and check each pair with `np.isclose`. (`np.corrcoef` returns a small table of correlations rather than a single number; its entry `[0, 1]` is  $r$ . That indexing is all you need to know about it.)

State in a comment under each function whether every value agrees with your hand computation. If any disagree, find the error — it is far more often in the hand computation than in the code.

**Problem 2** (Writing the dot product and the  $p$ -norm yourself).

- (a) Write `my_dot(u, v)` using a for loop over the indices — not `@`, `np.dot`, or `np.sum(u * v)`. It should raise a `ValueError` if the lengths differ. Return `my_dot` on the three pairs of Problem E.1(d), each checked against `@` with `np.isclose`.
- (b) Write `pnorm(v, p)` for  $p \geq 1$  and `infnorm(v)`, using only `np.abs`, `np.sum`, `np.max` and `**`. With `rng = np.random.default_rng(231)`, draw five random vectors in  $\mathbb{R}^7$  and, for each, compare `pnorm` with `np.linalg.norm(v, ord=p)` for  $p = 1, 2, 3, 4.5$ , and `infnorm` with `ord=np.inf`. Return `True` only if all 25 comparisons pass.
- (c) For `v = [-3, 4]`, return `pnorm(v, p)` for  $p = 1, 2, 3, 4, 10, 100$ , and `infnorm(v)`. Compare with Problem B.1(iii).
- (d) With `rng = np.random.default_rng(7)`, draw one random vector in  $\mathbb{R}^{10}$  and return `pnorm` for  $p = 1, 2, 4, 8, 16, 32, 64, 128$ , followed by `infnorm`. In a comment: at roughly which  $p$  is  $\|v\|_p$  within 1% of  $\|v\|_\infty$ ?

**Problem 3** (The norm inequalities at scale).

Problem E.7 proved, for  $v \in \mathbb{R}^n$ ,

$$\|v\|_\infty \leq \|v\|_2 \leq \|v\|_1 \leq \sqrt{n} \|v\|_2 \leq n \|v\|_\infty.$$

- (a) With `rng = np.random.default_rng(231)`, draw 10,000 random vectors in  $\mathbb{R}^{10}$  and count how many satisfy all four inequalities.
- (b) In the same run, return the largest and the smallest value of  $\|v\|_1/\|v\|_2$  you observed, next to the bounds 1 and  $\sqrt{10}$  that E.7 predicts.
- (c) Find a nonzero vector in  $\mathbb{R}^{10}$  for which  $\|v\|_2 \leq \|v\|_1$  is an equality, and one for which  $\|v\|_1 \leq \sqrt{n} \|v\|_2$  is an equality. Build both in code, and return `np.isclose` verdicts confirming it.
- (d) Answer in a comment. The random vectors came much closer to one of the two bounds in (b) than to the other. Which one? Using your vectors from (c), say what a vector must look

like to attain each bound, and why one kind is far rarer among random draws than the other. Finally: does the count in (a) prove E.7?

**Problem 4** (Similarity and nearest neighbours).

- (a) Write `cos_sim(u, v)`. For the documents of Problem E.4, return  $\cos(A, X)$  and  $d_2(A, X)$  for  $X = B, C, D$ .
- (b) For the depot problem B.2, return, for each of `ord=1`, `ord=2` and `ord=np.inf`, the name ("A", "B" or "C") of the nearest customer. Let the code find the minimum; do not type the answer in.
- (c) For nonzero  $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ , the unit vectors  $\hat{\mathbf{x}} = \mathbf{x}/\|\mathbf{x}\|$  and  $\hat{\mathbf{y}} = \mathbf{y}/\|\mathbf{y}\|$  satisfy

$$\|\hat{\mathbf{x}} - \hat{\mathbf{y}}\|^2 = 2 - 2\cos(\mathbf{x}, \mathbf{y}),$$

so once every document is normalized, ranking by distance and ranking by cosine similarity give the same order. With `rng = np.random.default_rng(231)`, draw 1000 pairs of random vectors in  $\mathbb{R}^{50}$  and check the identity on every pair with `np.isclose`. Return the number of pairs that pass.

**Problem 5** (Projections and coordinates in  $\mathbb{R}^n$ ).

- (a) Write `proj(u, v)`, returning `proj_u(v)`. Use it on Problem A.3(a) and on the pair of Problem D.3, and return each projection together with the dot product of its orthogonal piece against the vector projected onto.
- (b) Write `coords(basis, x)`, where `basis` is a Python list of mutually orthogonal nonzero arrays, returning the list of coefficients  $\langle \mathbf{b}, \mathbf{x} \rangle / \langle \mathbf{b}, \mathbf{b} \rangle$ . Write also `is_orthogonal_set(vectors)`, which checks every pair with `np.isclose`. Return the coordinates (and the orthogonality verdict) for every vector in Problems D.1(a) and D.1(c).
- (c) For each case in (b), rebuild the vector from its coordinates and return the `np.allclose` verdict against the original.
- (d) Now misuse `coords` on the basis  $\{[2, 1], [1, 3]\}$  of Problem C.1, which is not orthogonal, with  $\mathbf{x} = [5, 5]$ . Return the coefficients it produces, the vector they rebuild, and the `np.allclose` verdict. Explain in a comment which step of the derivation in vector unit §21.6 fails when the basis is not orthogonal.

**Problem 6** (Life in high dimensions).

- (a) Return the angle in degrees between  $\mathbf{1}$  and  $\mathbf{e}_1$  in  $\mathbb{R}^n$  for  $n = 2, 3, 4, 10, 100, 10000$ , computed from the vectors themselves (not from the formula  $1/\sqrt{n}$  of Problem E.6). Compare with E.6(b).
- (b) With `rng = np.random.default_rng(231)`, for each  $n = 2, 10, 100, 1000, 10000$  draw 2000 pairs of random vectors in  $\mathbb{R}^n$  and compute the average of  $|\cos(\mathbf{x}, \mathbf{y})|$  over the pairs. Return the five averages, and the five values of  $\sqrt{n}$  times the average.

- (c) Answer in a comment, in two or three sentences: what happens to the angle between two random vectors as  $n$  grows? Search engines and language models represent words and documents as vectors in  $\mathbb{R}^n$  with  $n$  in the hundreds or thousands and entries of both signs. If two such vectors have cosine similarity 0.3, is that a large number or a small one, and why? (Raw word counts, as in Problem E.4, behave differently: their entries are never negative, so their cosine is never negative, and the experiment in (b) does not describe them.)

**Problem 7** (The derivative as a limit, numerically).

- (a) Write `diff_quotient(f, x, h)`, returning  $(f(x+h) - f(x))/h$ . For  $f = \exp$  at  $x = 1$ , return the error  $|\text{quotient} - e|$  for each  $h = 10^{-1}, 10^{-2}, \dots, 10^{-15}$ , and the value of  $h$  that gave the smallest error.
- (b) Answer in a comment: the error shrinks as  $h$  shrinks, and then grows again. Explain both halves. (The Recipe Book’s “Watching cancellation”, §7, is the second half.)
- (c) Return  $(e^h - 1)/h$  and  $(1+h)^{1/h}$  for  $h = 10^{-1}, \dots, 10^{-8}$ . Which two numbers from the vector calculus lecture are they approaching?
- (d) With  $h = 10^{-6}$ , use `diff_quotient` to check your answers to Problem F.1(a), (b), (e) and (f) at  $x = 2$ : code your hand formula for each derivative, and compare with `np.isclose(..., rtol=1e-4)`. Return the four verdicts.

**Problem 8** (Partial derivatives and the gradient, numerically).

- (a) Write `partial_x(f, x, y, h=1e-6)` and `partial_y(f, x, y, h=1e-6)` from the limit definitions, and `grad(f, x, y)` returning the gradient as a NumPy array. Return `grad` of  $x^3 - 3xy + y^2$  at  $(1, 2)$ ,  $(2, -1)$  and  $(0, 0)$ , next to your hand values from Problem F.6(a), with `np.allclose` verdicts (use `atol=1e-4`).
- (b) Return  $\|\nabla f(2, -1)\|$  computed numerically, next to the exact value.
- (c) Check your answers to Problem F.4(b), (c) and (h) at the point  $(1, 0.5)$ : code your hand formulas and compare. Return the verdicts.
- (d) For  $f(x, y) = \sqrt{x^2 + y^2}$ , with `rng = np.random.default_rng(231)`, draw five random points and return  $\|\nabla f\|$  at each. Compare with Problem F.6(d).

**Problem 9** (Running ahead: gradient descent).

This problem uses §8.4 of the vector calculus unit, which has not been lectured yet. Everything you need is here:

#### From §8.4

**Gradient descent** minimizes a function  $f : \mathbb{R}^2 \rightarrow \mathbb{R}$  by repeatedly stepping in the direction of steepest descent,  $-\nabla f$ :

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k),$$

with a fixed step size  $\alpha > 0$  and a chosen starting point  $\mathbf{x}_0$ .

- (a) Write `descend(grad_f, x0, alpha, steps)`, returning the list of all points  $\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{\text{steps}}$ . For  $f(x, y) = x^2 + 4y^2$ , with  $\mathbf{x}_0 = (2, 1)$  and  $\alpha = 0.1$ , return  $\mathbf{x}_1$  and  $\mathbf{x}_2$ , and the point and the value of  $f$  after 50 steps.
- (b) Repeat with  $\alpha = 0.25$  and with  $\alpha = 0.3$ . Return the point and the value of  $f$  after 50 steps for each.
- (c) Answer in a comment. Show by hand that for this  $f$  one step is  $x_{k+1} = (1 - 2\alpha)x_k$  and  $y_{k+1} = (1 - 8\alpha)y_k$ . Use it to explain all three runs, and find the range of  $\alpha$  for which the iteration converges to the minimum.

Finally, answer in a comment at the end of your file, in three or four sentences: which problems on Homework 2 could NumPy check for you, and which could it not? Name at least one proof problem and say precisely what a computer run would and would not establish about it.

### What NumPy can and cannot do for you

| Homework 2            | Can NumPy check it?                                                   |
|-----------------------|-----------------------------------------------------------------------|
| A — projection        | A.1–A.3 and the numbers in A.4–A.6, yes; the proofs, no.              |
| B — norms and metrics | B.1 and B.2, yes; B.3–B.5, no.                                        |
| C — bases and span    | C.1–C.3 and C.7, yes; the proofs in C.4–C.6, no.                      |
| D — orthogonality     | D.1, D.2 and the numbers in D.3, yes; the proof in D.3, D.4, D.5, no. |
| E — $\mathbb{R}^n$    | The numbers, yes; E.2, E.3(c), E.5(a), E.7, no.                       |
| F — derivatives       | Every derivative at a point, yes; F.3, F.6(c)–(d), no.                |
| This assignment       | It <i>is</i> the check.                                               |

A numerical derivative checks a formula at the points you try. It cannot tell you the formula is right everywhere — but a mismatch at one point proves the formula wrong.

### Optional and ungraded: drawing the pictures

Not required, not graded, and not needed for any later assignment. It needs `matplotlib`, as in Homework 1:

```
python -m pip install matplotlib
```

**Watch out.** Keep this out of `ec1.py` and `runec1.py`, in a scratch file of its own. `plt.show()` halts a program until you close the window, so a plot left in your submission stalls the run it is graded from.

The unit balls of vector unit §14.8, for several values of  $p$ . Each point of the ordinary unit circle is rescaled to have  $p$ -norm 1:

```
import numpy as np
import matplotlib.pyplot as plt
```

```

t = np.linspace(0, 2*np.pi, 400)
x, y = np.cos(t), np.sin(t)
for p in [1, 1.5, 2, 4, 10]:
    size = (np.abs(x)**p + np.abs(y)**p) ** (1/p)    # the p-norm of each point
    plt.plot(x / size, y / size, label=f"p = {p}")

plt.gca().set_aspect("equal")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

The level curves of  $g(x, y) = x^2 + 4y^2$  — the function Problem 9 minimizes — with its gradient drawn at a grid of points. `np.meshgrid` builds the grid of points; you do not need to understand the shape of what it returns to use it.

```

xs = np.linspace(-2, 2, 200)
X, Y = np.meshgrid(xs, xs)
plt.contour(X, Y, X**2 + 4*Y**2, levels=12)

xg = np.linspace(-2, 2, 9)
XG, YG = np.meshgrid(xg, xg)
plt.quiver(XG, YG, 2*XG, 8*YG, color="C3")    # the gradient [2x, 8y]

plt.gca().set_aspect("equal")
plt.show()

```

Notice that the arrows cross every level curve at a right angle, and that they do not point away from the centre except along the two axes.

---

Questions about any problem here belong in the class Discord, where everyone can see the answer. Questions that are specific to your own work — “why does my `runec1.py` stop at Problem 4?” — are better brought to office hours, Thursdays 3:00–4:00 PM in Kiely Hall 508. Please email ahead.